
Knowing What You Need Is Not Enough: The Policy Bottleneck in Homeostatic Agents

Anonymous Author(s)
Anonymous Institution
anonymous@email.com

Abstract

1 A homeostatic agent acts to keep internal drives near their setpoints, so its behaviour
2 hinges on the *satiation signal* g : how much each outcome quiets each drive. Hand-
3 designing g risks an agent that regulates flawlessly toward the wrong thing — the
4 Satiation Signal Problem. We ask whether g can instead be learned from observable
5 consequences, in a domain anyone can picture: a student with forty study sessions
6 before an exam, choosing how hard to push, pulled between wanting to progress
7 and wanting to feel competent. Across a dose–response sweep of environmental
8 adversity, with predictions hashed before execution and paired analysis over 100
9 student profiles, we decompose the gap between the homeostatic agent and a
10 simple state-estimating baseline. Signal quality is the smallest term: a *perfect* g
11 buys 7.4% of the gap, re-tuning six policy coefficients buys 27.7%, and the rest is
12 neither. Giving each signal its own tuning budget, across five independent searches
13 validated on held-out profiles, the ordering *is* monotone — a perfect g beats a
14 constant in five of five — but the advantage is about 0.05 against a remaining
15 distance to the estimator of 0.4–0.6, and a *learned* g buys essentially nothing over
16 a well-chosen constant. Knowing what satisfies the agent helps, and is nowhere
17 near enough; the channel that turns satiation into action is what binds. Homeostatic
18 regulation still earns a role, but narrower than we first claimed: it buys viability
19 rather than optimality, and only against baselines lacking any rest mechanism.
20 Diagnosing why, we find our own applicability framework was under-specified —
21 and propose a sixth condition, tested post-hoc, under which the ordering reverses.

22 1 Introduction

23 What moves an agent to act? In biology, action is not a default state — it emerges when internal
24 variables leave viable ranges [1]. Yet dominant paradigms in autonomous agent design optimize
25 external objectives and rarely treat internal regulatory integrity as a first-class concern. What the
26 agent itself *needs* is systematically absent.

27 That absence shows in where engineering effort has gone: from prompt engineering to context
28 engineering to harness and loop engineering, successive scaffolding layers around a capable model.
29 Each adds capability; none adds motivation. This line of work bets on a different move — from
30 **agent engineering** to **agent cybernetics**: less effort adding capabilities, more inserting the regulatory
31 structure that decides when and why a capability should be exercised. It matters because deployment
32 surfaces the failure modes capability alone does not address — agents that over-optimize, accumulate
33 context until collapse, or pursue goals long after they stop being appropriate. Autonomy is the
34 capacity to *remain viable while doing*, not just to *do*.

35 Prior work in this line established feasibility. The Pro-Action Operator Γ showed that coupled
36 homeostatic drives integrated with LLM reasoning produce behaviour differentiated by opponent type
37 in iterated social dilemmas (ICML 2026, anonymized). A single-drive variant produced emergent

38 regulatory behaviour in debate facilitation without any reinforcement learning — and surfaced a
39 structural limitation: g was hand-designed, and when misaligned the system regulated toward the
40 wrong attractor (SANLP 2026, anonymized). Tied to a structural metric it regulated participation but
41 not semantic fidelity; tied to an LLM judge, context collapse *quintupled*. The loop worked — toward
42 the wrong thing.

43 We call this the **Satiation Signal Problem (SSP)**: how is g defined so the system regulates toward
44 a desired external goal? It is one problem inside the broader agenda of agent cybernetics, not the
45 agenda itself — but it gates the rest, because the mechanism is value-neutral and the signal feeding it
46 decides what it regulates toward. So we ask: **can a homeostatic agent learn g from the observable
47 consequences of its own actions, and use it to regulate toward an externally useful goal?** The
48 domain is a student with forty study sessions before an exam, choosing difficulty guided by two
49 drives — one pushing, one braking — while never observing its own ability, fatigue or frustration.

50 A single operating point cannot answer this: an earlier iteration produced a null admitting two
51 incompatible readings — the signal does not matter, or the environment never applied pressure
52 enough to make it matter. We therefore sweep the pressure, hash the predictions before execution,
53 gate every sweep point, and analyze 100 profiles as the paired design it is.

54 Our central result is a decomposition rather than a verdict: of the gap to a state-estimating baseline, a
55 perfect oracle g buys 7.4%, re-tuning six hand-chosen coefficients buys 27.7%, and the rest is neither.
56 Giving each signal its own tuning budget — five independent searches, validated on 50 profiles never
57 used for tuning — the ordering is monotone in signal quality, but by only ≈ 0.05 against a remaining
58 distance of 0.4–0.6, and a *learned* g is indistinguishable from a constant. Knowing what you need
59 helps; it is nowhere near enough. Where regulation earns a role is viability, and there too we narrow
60 an earlier claim: the homeostatic agent with rest disabled still reaches 93.5% dropout against the
61 estimator’s 100%, so viability is bought by having *some* rest mechanism.

62 Four contributions follow: the decomposition, with paired effect sizes and confidence intervals and
63 replicated on the goal metric; the matched-tuning result, which bounds how much the SSP is worth
64 solving, since a *learned* signal does not beat a constant; a methodology — dose–response sweeps
65 with per-point gating, hashed predictions, multiplicity-corrected existence tests, and multi-seed tuning
66 with held-out validation — for architectures making claims about *regimes*; and a revision of our own
67 applicability framework, including a sixth condition under which the ordering reverses. We also
68 report, in the body rather than a footnote, three of our own claims that stricter protocols forced us to
69 withdraw.

70 2 Background and Related Work

71 In game AI, Zubek [8] synthesized the needs-based approach exemplified by The Sims — drawing on
72 robotics and his own industry practice rather than on that title’s implementation — in which agents
73 act to satisfy a vector of decaying needs. It produces believable behaviour without scripting, but the
74 mappings are hand-authored: the agent never discovers what satisfies it; it is told. What if it had to
75 learn them?

76 That question leads to biology. Cannon [1] formalized homeostasis as maintaining internal variables
77 within viable ranges through negative feedback — an architecture robust enough to underpin tem-
78 perature, glucose, and fluid regulation across all vertebrates, but one that corrects *after* deviation
79 is detected. Sterling and Eyer [2] extended this with allostasis: predictive adjustment based on
80 anticipated demand. An allostatic system does not wait for glucose to drop. The distinction is
81 temporal, and it predicts a failure mode our agent encounters.

82 Keramati and Gutkin [3] bridged these principles with RL, defining primary reward as the reduction
83 in homeostatic deviation — proportional to $D(H_t) - D(H_t + K_t)$, the drive before and after an
84 outcome — and proving that maximizing discounted reward is equivalent to minimizing discounted
85 deviation, so that reward-seeking *is* stability-seeking. Yet the drive function and its parameters are
86 fixed: the agent learns *which actions* reduce deviation, not *how much each outcome satisfies each*
87 *drive*. That mapping is prescribed. Our work picks up exactly there.

88 The gap matters beyond HRL. The successor representation literature [5–7] showed that agents
89 benefit from modelling how the environment affects future states — which our results identify as the
90 necessary next step. Ashby’s Law of Requisite Variety [4] constrains what our agent can achieve, and

our residual term is best read as a variety failure located not in the drives but in the channel connecting them to action. A further critique, the Rule-Relocation Problem (various authors, 2025–2026), notes that the setpoint is itself a designer-imposed norm: normativity is relocated from reward function to setpoint rather than eliminated. The SSP is a special case, g being the relocated norm.

3 When Is a Problem Regulatory?

A framework that cannot say where it does *not* apply is not a framework. We hold that a problem is regulatory when it exhibits five properties:

1. **Variables that must be sustained within viable ranges** — not maximized, but kept in a band over time. Sleep is the everyday case: neither zero nor sixteen hours beats seven, and the cost of leaving the band compounds.
2. **Antagonistic tensions that must be held without collapse** — satisfying one need destabilizes another, and the trade-off cannot be resolved once and forgotten. A household budget balances spending against saving continuously; training balances stimulus against recovery.
3. **Non-linear dynamics** — crossing a threshold has consequences that do not smoothly extrapolate. Overtraining does not degrade performance gradually; it produces an injury.
4. **Situated information particular to the agent** — the agent observes its own condition in ways an external orchestrator cannot. A coach sees your lifts; only you feel the joint about to give.
5. **An objective learning signal** — outcomes are measurable without human judgment or circularity.

These five are our starting position; Section 7.4 revises them in light of what the experiment showed, including a sixth we did not anticipate. The self-regulated learner satisfies (5) unambiguously: an answer is correct or it is not. Conditions (1)–(4) are what our experiment manipulates, and a central methodological finding here is that they must be manipulated rather than assumed. The everyday version: push to your maximum every day from the start and you quit; never push and you stagnate. The right behaviour is not a schedule but a dynamic balance between the drive to improve and the need to recover, adjusted from how your body responds.

A note on the target band. We measure the difficulty band that maximizes expected learning gain in the simulator. This is inspired by, but narrower than, Vygotsky’s Zone of Proximal Development [9], which concerns the gap between independent and assisted performance. Our environment models no scaffolding, so we call the measured quantity the **optimal-challenge band** and use time-in-band (TIB) as its metric, reserving the ZPD label for the motivating construct.

4 Architecture

4.1 Two drives, and what they feel like

Everything below follows from one design choice: the agent is given *needs*, not goals — two of them, both deficits that grow until something quiets them.

The first, δ_{prog} , is the itch of coasting — what you feel in the third week of lifting the same comfortable weight: nothing is going wrong, and that is exactly the problem. It grows as sessions pass, faster the more you are already succeeding, and is quieted only by genuine progress. Left alone it pushes upward.

The second, δ_{conf} , is the flinch after a bad run — what makes you re-rack the bar after two failed lifts. It jumps on errors and is quieted by easy successes. Left alone it pulls downward.

Neither drive is a goal, and neither knows anything about difficulty levels. The agent never observes its ability, fatigue or frustration — only which level it chose, whether it was right, and how long it took. Behaviour is whatever falls out of two deficits pulling opposite ways.

136 4.2 Why two drives and not one

137 A single drive cannot tell the two failure modes apart: if the agent is dissatisfied, is the material too
 138 easy or too hard? A lone drive registers only “not satisfied” either way. This is Ashby’s requisite
 139 variety [4] in concrete form, and two antagonistic drives are the minimum.

140 Formally, each drive i has a setpoint θ and evolves per session as

$$\delta_{i,t+1} = \delta_{i,t} + \lambda_i m_i - \alpha_i g_i[k] \mathbf{1}[\text{satiating outcome}] + w_{ij} p_i(\delta_i, \theta) \mathbf{1}[\delta_j > \theta + \epsilon] \quad (1)$$

141 reading left to right: the drive grows by a basal λ_i scaled by observable mastery m_i (hunger between
 142 meals); it shrinks by $\alpha_i g_i[k]$ when a satisfying outcome arrives at level k ; and the last term couples
 143 the drives, active only when the other is above setpoint. That coupling is modulated by the *receiving*
 144 drive’s own pressure, $p(\delta, \theta) = \min(1, |\delta - \theta|/0.8)$, so a drive at setpoint ignores its counterpart
 145 while one already under strain responds sharply — the reason a warning lands differently depending
 146 on how close to your limit you are. Pressure is symmetric because an under-challenged learner is not
 147 content, they are bored, and boredom generates its own pressure to act.

148 4.3 The satiation signal, and why it is the crux

149 The term $g_i[k]$ is where the design problem lives: it decides how much a correct answer at level k
 150 counts as progress. Set it wrong and the machinery still works perfectly — it just regulates toward
 151 the wrong place. Fix g to a constant and every correct answer, however trivial, quiets the progress
 152 drive equally; the agent settles at the easiest level it can find and calls itself satisfied. That is the SSP
 153 in one sentence.

154 The learning rule uses only what the agent can see — a running accuracy per level, $\text{acc}[k]$:

$$g_{\text{prog}}[k] \leftarrow (1 - \eta) g_{\text{prog}}[k] + \eta \cdot 4 \text{acc}[k] (1 - \text{acc}[k]), \quad g_{\text{conf}}[k] \leftarrow (1 - \eta) g_{\text{conf}}[k] + \eta \cdot \text{acc}[k] \quad (2)$$

155 The intuition is ordinary: a level where you get everything right teaches little, and so does one where
 156 you get everything wrong. A mastered level yields low g_{prog} , correct answers barely satisfy the
 157 progress drive, it keeps growing, and the agent climbs. An impossible level also yields low g_{prog}
 158 while errors inflate confidence, and the agent retreats. In between, satiation balances growth. **The**
 159 **target band is never encoded — it is where the dynamics come to rest.**

160 One property bounds everything that follows: $4a(1 - a)$ is symmetric with its maximum at $a=0.5$,
 161 whereas expected gain in our environment is maximized at a difficulty gap of 0.40, where accuracy is
 162 only 0.354. The rule’s peak sits on the wrong side of the true optimum, capping achievable fidelity
 163 regardless of how much data the agent collects.

164 4.4 From drives to actions, and what that costs

165 Something must convert two numbers into one of four choices: go up, stay, go down, or rest. We use
 166 four linear logits over the drive deviations d_p and d_c with six coefficients, sampled by softmax, with
 167 REST gated by normalized frustration; ablation arms replace sampling with argmax, remove the rest
 168 gate, or re-tune the coefficients (Appendix B). We flag this stage because it, not the satiation signal,
 169 is where the architecture loses — and its six coefficients were chosen by hand, which we treat as a
 170 variable to measure rather than a detail to defend.

171 The operator itself is deliberately cheap: per session, $O(n^2)$ coupling updates, $O(A)$ logits and $O(1)$
 172 signal updates — about twenty floating-point operations at $n=2$, $A=4$, $K=5$ — with $O(n^2 + K)$
 173 state, no gradient, no value function and no replay. Measured cost is 14 μs per decision.

174 5 Experimental Design

175 5.1 The environment, and the two things that make it a test

176 The simulated student has a hidden ability h ; correctness at level k follows a Rasch model [10],
 177 $p = \sigma(1.5(h_{\text{eff}} - k))$. Learning peaks at a moderate stretch above current ability and falls off both

ways, and failing at something challenging still teaches, so playing safe is not trivially optimal. Effort accumulates and drags down both effective ability and the capacity to learn; recovery slows sharply past a threshold, so exhaustion cannot simply be slept off. Resting costs one of the forty sessions and induces mild forgetting. Frustration rises with errors, rises further when failing at something previously mastered, falls with easy successes, and if sustained near a personal threshold for three sessions the student quits for good. The exam is scored on effective ability: you sit it in the state you arrive in.

Two details are load-bearing. Fatigue must carry a *deferred* cost, because an estimator updating from outcomes already tracks current fatigue for free — an accumulator earns its keep only if strain that is cheap now is expensive later. And quitting must be reachable, or viability is not a live variable. An earlier version of this environment failed both, which is why it produced an uninterpretable null.

5.2 Turning the adversity up, rather than picking one setting

The primary axis is ϕ , the severity of non-linear fatigue, over $\{0, 0.3, 0.6, 0.9, 1.2, 1.6\}$. Turning it up makes exhaustion bite, which produces errors, then frustration, then reachable quitting — one knob activates both the non-linearity and the viability pressure. A second axis moves the *peak* of the learning curve mid-episode, leaving any agent that models the curve misspecified in a way re-estimation cannot repair (Section 6.6).

Sweeping raises its own problem: at extreme adversity an environment can stop discriminating between good and bad policies. We therefore re-run a quality gate at *every* point — hiding at level 1 must stay unprofitable, the quitting boundary must bite, an oracle policy must beat random. It passes for $\phi \leq 1.2$ and fails at $\phi \geq 1.6$, where severe fatigue makes hiding viable and the trap dissolves. Because that exclusion is a judgement call, we repeat every headline contrast with the excluded point included (Appendix C).

5.3 Committing to predictions before looking

A sweep with ten arms and six points offers many places to find a story after the fact. The predictions below were therefore written as plain text and hashed with SHA-256 before any sweep executed:

```
6e888c7e46e1dd306b6adc601a45cc08ff914a24d2f474a913a1ff2907bfc084
```

Anyone can recompute the digest from `predictions.txt` in the supplement. It is self-attested, not a third-party registry: it makes post-hoc editing detectable, nothing more. The gate thresholds were *not* in the committed text.

- **P1 (viability).** Estimation without viability management collapses as ϕ rises.
- **P2 (robustness).** Γ arms degrade less than the estimator across the sweep.
- **P3 (crossover).** Some Γ arm overtakes the estimator within the valid range.
- **P4 (policy).** Argmax beats softmax throughout.
- **P5 (signal).** If g 's quality matters behaviourally, oracle separates from a fixed constant.
- **P6 (learnability).** g validity decreases as adversity rises.
- **P7 (misspecification).** Under a change in the *shape* of the learning curve, Γ arms gain relative ground against the now-misspecified estimator.

5.4 Arms and how they are compared

Each arm isolates one suspect. Γ_{oracle} , Γ_{fixed} and Γ_{naive} vary only the satiation signal; Γ_{argmax} and $\Gamma_{\text{restdrive}}$ vary only the policy; rule and estimator are non-homeostatic baselines, each run with and without its rest mechanism so viability separates from level selection. The estimator is a ten-line agent that tracks ability from outcomes and picks the level its model says is best.

100 profiles $\times$ 10 seeds $\times$ 10 arms $\times$ 6 points = 60,000 episodes. Every profile faces every arm, so contrasts use **paired t -tests on profile-level means** ($n=100$) with 95% CIs and d_z — an earlier analysis of ours used independent-samples tests here and reported a null that pairing overturns.

Two precautions follow. Because several conclusions rest on the *absence* of an effect, we run TOST equivalence tests against $\delta=0.05$, with sensitivity across $\delta \in [0.02, 0.10]$. And because P3 is an *existence* claim over arms and adversity points, we evaluate it with a max-statistic sign-flip permutation test over all arm-by- ϕ cells rather than quoting the winning one.

6 Results

6.1 Main sweep

Across the valid range the estimator falls from +1.132 to +0.156 while Γ_{learned} moves only from +0.248 to +0.143; dropout stays at zero for every arm that can rest and climbs from 16% to 100% for the estimator that cannot (Appendix D).

P1 confirmed. `estimator_norest` rises from 16% to 100% dropout: estimating your state perfectly is worthless if you quit in session 20. **P2 confirmed**, and tested as an interaction rather than a ratio of aggregates: fitting each profile’s own gain-versus- ϕ slope, the estimator falls at -0.808 per unit ϕ while Γ_{learned} falls at -0.070 , a paired difference of $+0.737$, CI $[+0.706, +0.769]$, $d_z=+4.65$. The regulated agent is flatter, not merely lower.

6.2 Decomposing the gap: signal, policy, residual

The estimator leads by $+0.860$ at $\phi=0$. We decompose that gap with a 2×2 over signal quality (constant vs. oracle g) and policy parameterization (hand-chosen vs. re-tuned coefficients; Appendix L). The two manipulations interact, so we report all four components, summing exactly (Appendix E): signal $+0.063$ (7.4%), policy $+0.238$ (27.7%), interaction -0.047 (-5.5%), residual $+0.606$ (70.5%). Repeated on time-in-band — the metric that operationalizes the stated goal — the shares are 8.7, 29.4, -9.7 and 71.7%, so the conclusion is not an artefact of measuring terminal ability instead of the target band. Against the model-free estimator of Section 6.4, arguably the fairer comparator, the gap is $+0.580$ and the shares become 10.9, 41.0, -8.1 and 56.0%: the ordering is unchanged, only the magnitudes move.

6.3 Matched tuning: signal quality does order performance, weakly

The shared coefficients above were optimized against stable signals, so applying them to a noisy learned signal is not a fair test — a point we owe to review. We therefore repeated the search per signal. A first version used one search each and validated on the profiles it had tuned on; it produced a non-monotonic ordering in which a learned g beat a perfect one, which we initially reported as the headline. It does not survive a stricter protocol: **five independent searches per signal** (identical 250-candidate budget), validated on **50 profiles never used for tuning**. The held-out means are $+0.472$ (sd 0.051) for a constant g , $+0.480$ (sd 0.068) for a learned one, and $+0.519$ (sd 0.052) for the oracle; a single larger run at 1200 candidates gives $+0.411$, $+0.545$ and $+0.656$ (Appendix K). For reference the estimator scores $+1.132$, and $+0.851$ without its objective model.

Three things follow. **Signal quality does order performance**: the oracle beats a constant in five of five searches, and the gap widens at a larger budget — sparse search was masking the effect, and correcting for it strengthens signal quality rather than eliminating it. **Learning g buys essentially nothing over a well-chosen constant** ($+0.480$ vs. $+0.472$), which bounds how much effort the SSP repays. And the effect is small in the relevant units: roughly 0.05 against a remaining distance of 0.4–0.6. Knowing what you need helps, and is nowhere near enough. The 1200-candidate column is a single exploratory run; the drop in its constant- g cell is the tuning-set overfitting we warn about elsewhere.

6.4 Is the estimator winning because it knows the objective?

The estimator does more than track ability: it also knows which level maximizes learning gain — privileged knowledge of the simulator’s objective, which might explain the gap by itself. We therefore added an estimator that keeps ability tracking but discards that model, matching the level to its ability estimate. Removing it costs the estimator in the benign regime ($+1.132 \rightarrow +0.851$) but *helps* it under adversity, where the model is misspecified ($+0.156 \rightarrow +0.275$ at $\phi=1.2$; Appendix H).

273 So the answer is no: stripped of its objective model the estimator stays three to six times better
 274 than Γ . But this baseline costs us a claim. **The crossover does not survive against it** — at $\phi=1.2$
 275 the model-free estimator scores +0.275 against Γ_{argmax} 's +0.201. We therefore withdraw the
 276 crossover as evidence that homeostatic regulation overtakes state estimation, and retain it only as
 277 evidence that a *misspecified* objective model degrades faster than drive-based regulation.

278 6.5 Signal quality across adversity, and the crossover

279 **P5 is confirmed, with a boundary** (Appendix F). At $\phi=0$ a perfect signal is worth +0.063
 280 ($d_z=+1.59$, $p=6 \times 10^{-29}$); from $\phi=0.3$ onward the difference falls inside the equivalence bound
 281 at every margin from 0.02 to 0.10 — practical equivalence, not a proven null. This corrects an
 282 earlier analysis of ours that applied independent-samples tests to this paired design and reported no
 283 separation anywhere: a Type II error from discarding the within-subject structure.

284 **P4 confirmed**, with the effect growing monotonically from $d_z=+0.21$ at $\phi=0$ to $d_z=+3.23$ at
 285 $\phi=1.2$ (full contrasts in Appendix G).

286 **P3 confirmed against the model-based estimator** — though not against the stronger one of
 287 Section 6.4 — with three qualifications. At $\phi=1.2$ the regulated agent overtakes by +0.045, CI
 288 [+0.028, +0.062]; since P3 is an existence claim, a max-statistic sign-flip permutation over all Γ
 289 arms and valid ϕ gives $p=.0005$ (max $t=+5.18$ at that cell), so it survives multiplicity correction.
 290 **But the crossover belongs to the Γ_{argmax} ablation, not the main agent:** Γ_{learned} scores
 291 +0.143 against +0.156 and does not overtake. And +0.045 sits below our own $\delta=0.05$ threshold —
 292 a robust reversal of ordering, not a large one. Excluding $\phi=1.6$ was conservative: including it the
 293 contrast grows to +0.089, $d_z=+1.54$.

294 The argmax contrast is not a clean isolation of sampling noise: changing the action rule moves the
 295 entire closed loop — visitation, fatigue, rest frequency, and the resulting g estimates. Γ_{argmax}
 296 attains higher signal validity (+0.604 vs. +0.471) under an identical learning rule, because systematic
 297 visitation yields cleaner estimates. Read P4 as “the deterministic policy is better,” not as “sampling
 298 noise costs this much.”

299 **P6 confirmed but policy-dependent.** Validity for Γ_{learned} falls from +0.471 to +0.113; for
 300 Γ_{argmax} it remains +0.448 at $\phi=1.2$. Learnability is therefore not bounded by non-stationarity as
 301 such — it is bounded by non-stationarity *under a stochastic policy*.

302 6.6 Model misspecification, not situated information

303 Moving the learning curve's peak mid-episode leaves the estimator's model wrong in a way re-
 304 estimation cannot fix. **P7 is refuted:** the gap does not shrink but *widens*, from -0.181 to -0.198
 305 ($d_z -1.02$ to -1.26 , all $p < 10^{-16}$; Appendix M) — at the largest shift the estimator picks a level
 306 yielding roughly 13% of achievable gain and still wins threefold. This does *not* refute situated
 307 information, and we were too quick to say it did: the manipulation corrupts the competitor's model
 308 rather than granting Γ a private signal, and Γ 's own rule is anchored at $a=0.5$, so when the peak
 309 moves the estimator's fixed assumption ends up *closer* to truth than Γ 's. Condition (4) therefore
 310 **remains untested.**

311 7 Discussion

312 7.1 Did we answer the question we asked?

313 *Can a homeostatic agent learn g from observable consequences?* Yes — validity +0.471 against
 314 a +0.996 ceiling. *Can it use that signal to regulate toward the external goal?* Only marginally: a
 315 perfect signal buys under a tenth of the distance to a simple estimator while a rigid policy buys nearly
 316 four times more, and a *learned* signal is indistinguishable from a well-chosen constant. The satiation
 317 signal is neither the bottleneck nor irrelevant; it is the smallest of the terms.

318 7.2 Is the homeostatic mechanism buying viability?

319 Our most confident claim, and it did not survive. The comparison behind it pitted the regulated agent
 320 against an estimator with *no* rest mechanism — only one side could rest. Disable rest on both and the

estimator hits 100% dropout, Γ 93.5%: the predicted direction, not a difference in kind. Restore rest and both sit near zero, with the estimator also ahead on gain. Viability is bought by having *some* rest mechanism; a hand-coded rule recovers nearly all of it.

We also claimed regulation folds stopping into the machinery that picks difficulty. Appendix B says otherwise: the default agent’s rest logit is frustration-gated, so the drives barely participate. The claim holds only for *restdrive* — which pays 4–17% dropout for it.

7.3 Our framework says this is a regulatory problem. Why did regulation lose?

Section 3 lists five conditions under which we expect regulation to pay. Does this domain satisfy them? If it does, the framework is in trouble.

Conditions (3) and (5) hold cleanly, and (3) delivers the predicted signature: across the sweep the estimator loses 86% of its gain, Γ only 42%. Condition (1) holds *cheaply* — quitting is reachable, but a one-line guard erases it. Condition (2) holds nominally: the estimator has no confidence drive and still wins, so the antagonism is not irreducible.

Condition (4) fails outright, and we can price it. An outside observer of the outcome stream alone reconstructs the agent’s effective ability to within 0.32, on a scale where levels sit 1.0 apart. Nothing is private.

That is the crux. Conditions (1), (2), (3) and (5) say regulation is *needed*; only (4) says the agent must be the one doing it. It is load-bearing, and we treated all five as equals. Tellingly, Γ gains ground exactly where (4) begins to hold: as adversity drives reconstruction error from 0.32 to 0.86, the estimator collapses and Γ does not.

7.4 Is a sixth condition missing?

We think so, and it explains our result better than the other five. Homeostasis is a *maintenance* formalism: no horizon, no notion that an exam falls on session 40. Our task has a terminal objective, and accumulated gain can be banked, so more is always better. We were asking a regulator to maximize a payoff. Hence a sixth condition: **the objective must be the maintenance itself, not a terminal payoff that maintenance merely enables.**

What would we expect if that is right? Replace the terminal objective with one that *cannot* be banked — mean readiness, the score you would get if the exam were today, averaged over every session — and the ordering should reverse. It does, under a conjunction of three. At our own setting ($\phi=0.6$, 40 sessions) Γ still loses ($d_z=-1.07$). Stretch the horizon to 400 and it passes the model-based estimator ($+0.076$, $d_z=+2.09$). Add severe adversity and it beats *both* ($+0.044$ and $+0.031$, $d_z=+0.73$ and $+0.61$) while dropping out more often — so among survivors the margin is wider still (Appendix J).

A caution: post-hoc test of a post-hoc diagnosis, outside the committed predictions. But it could have failed, and a first version did. Merely lengthening the horizon, with gain still accumulative, changed nothing. Duration was never the variable. Bankability was.

7.5 What the methodology caught

Three of our own claims fell here, each to one methodological choice. An unpaired test on a paired design hid a real signal effect. A single tuning search invented a non-monotonic ordering that five searches erased. An unablated baseline let us credit homeostasis for what a rest rule was doing. Regulatory architectures make claims about *regimes* and are evaluated within-subject: dose–response sweeps, paired inference, multi-seed tuning with held-out validation, and ablation of the *baselines* — not only the proposal — should be default.

8 Limitations

The crossover is small, not the main agent’s, and does not survive the stronger baseline (Sections 6.4, 6.5). At $\phi=1.2$ all Γ arms lie between $+0.14$ and $+0.20$ against $+1.13$ for the estimator at $\phi=0$: Γ wins by degrading less, not by performing well.

367 **Signal validity is incompletely operationalized:** only g_{prog} , only against terminal expected gain, only
 368 as a five-point correlation, with g_{conf} unvalidated — so “learning g ” means learning g_{prog} . Relatedly,
 369 $4a(1 - a)$ peaks at $a=0.5$ while expected gain is maximized where accuracy is 0.354, so fidelity is
 370 capped by functional form and the oracle arm feeds the policy a differently shaped signal than the
 371 policy was tuned for.

372 **Policy tuning was a random search.** 250 candidates show the original coefficients were poorly
 373 chosen; they do not find the best policy. The residual therefore also absorbs unsearched policy
 374 space, the fixed action representation, and any joint signal–policy optimum we missed. It is the gap
 375 unexplained by *the interventions we tested*, not a structural constant.

376 **Researcher degrees of freedom remain:** a self-attested hash, gate thresholds outside the committed
 377 text (though including the excluded point strengthens the contrast), and a post-hoc sixth condition.
 378 Condition (2) is untested. The simulator is simple, the domain single-task, and six visits per level is
 379 thin for estimating g — itself one of our findings.

380 9 Future Work

381 The decomposition sets the priority: with most of the gap unexplained by signal or coefficients, the
 382 next step is to **learn the policy**, not the signal. We are non-committal about the method — prior
 383 work in this line found value-based RL integrated poorly with homeostatic drives, for reasons we do
 384 not yet understand — so direct policy search over the coefficient vector is the conservative first step,
 385 with evolutionary methods natural given how few parameters the operator has. Since matched tuning
 386 shows signal and policy must be co-adapted, they should be searched jointly, and condition (4) needs
 387 a test granting Γ a genuinely private signal.

388 A third question follows from our own null: if a learned g matches but does not beat a constant, the
 389 culprit may be that it *keeps* learning. A constant has zero variance; a signal estimated from six visits
 390 per level injects noise into the drives at every step, indefinitely. Would a **convergence-stopped** signal
 391 — annealing $\eta \rightarrow 0$, or freezing g once its moving average stabilizes — keep the bias reduction while
 392 shedding the variance?

393 Most consequential is allostasis. Homeostasis corrects after deviation; allostasis anticipates it [2]. The
 394 jump demands a **world model**: a learned predictor of environmental dynamics and of how actions
 395 propagate through them, rolled forward before committing. Not a self-model — the regularities live
 396 in the world. Harder material produces errors at a rate set by the gap to ability. Effort compounds
 397 non-linearly. Recovery has a threshold. The agent authors none of these laws and must infer them;
 398 its drives are merely where they register. Knowing your own state predicts nothing unless you also
 399 know how the world will act on it. That is what the Good Regulator Theorem demands, and the
 400 system to model is the coupled pair, not the agent alone. World model as mechanism, feedforward as
 401 implementation, allostasis as principle — and our residual is precisely what it would have to explain.

402 10 Conclusion

403 Can a homeostatic agent learn its satiation signal from observable consequences and use it to regulate
 404 toward an external goal? Analyzed as the paired design it is: signal quality matters, and matters least.
 405 A perfect signal is worth 7.4% of the gap to a simple estimator, six policy coefficients 27.7%, and the
 406 rest is neither. Under matched tuning the ordering does track signal quality — by about 0.05, while a
 407 *learned* signal is indistinguishable from a constant and the estimator stays ahead even stripped of its
 408 objective model.

409 Diagnosing why sent us back to our own framework, and we found it under-specified. Of five
 410 conditions, only one — that the agent hold what an outside observer cannot — makes homeostatic
 411 architecture necessary rather than merely applicable, and our domain fails it. We add a sixth: the
 412 objective must be maintenance, not a terminal payoff. Restore both and the ordering reverses.

413 Knowing what you need is not what limits you. The open problem is the channel, not the signal.

414 **References**

415 **References**

- 416 [1] **Cannon, W.B.** (1932). *The Wisdom of the Body*. New York: W.W. Norton.
- 417 [2] **Sterling, P. & Eyer, J.** (1988). Allostasis: A new paradigm to explain arousal pathology. In S. Fisher & J.
418 Reason (eds.), *Handbook of Life Stress, Cognition and Health*, pp. 629–649. New York: Wiley.
- 419 [3] **Keramati, M. & Gutkin, B.S.** (2014). Homeostatic reinforcement learning for integrating reward collection
420 and physiological stability. *eLife*, 3, e04811.
- 421 [4] **Ashby, W.R.** (1952). *Design for a Brain*. London: Chapman & Hall.
- 422 [5] **Dayan, P.** (1993). Improving generalization for temporal difference learning: The successor representation.
423 *Neural Computation*, 5(4), 613–624.
- 424 [6] **Gershman, S.J., Moore, C.D., Todd, M.T., Norman, K.A., & Sederberg, P.B.** (2012). The successor
425 representation and temporal context. *Neural Computation*, 24(6), 1553–1581.
- 426 [7] **Lehnert, L. & Littman, M.L.** (2019). Successor features combine elements of model-free and model-based
427 reinforcement learning. *Journal of Machine Learning Research*, 20(1), 1–53.
- 428 [8] **Zubek, R.** (2010). Needs-Based AI. In A. Lake (ed.), *Game Programming Gems 8*, pp. 231–238. Charles
429 River Media.
- 430 [9] **Vygotsky, L.S.** (1978). *Mind in Society: The Development of Higher Psychological Processes*. Cambridge,
431 MA: Harvard University Press.
- 432 [10] **Rasch, G.** (1960). *Probabilistic Models for Some Intelligence and Attainment Tests*. Copenhagen: Danish
433 Institute for Educational Research.

A Environment specification

Ability follows a Rasch model [10], $p = \sigma(1.5(h_{\text{eff}} - k))$. Learning gain per exercise is

$$\Delta h = 0.08 \exp\left(-\frac{(\text{gap} - 0.5)^2}{0.5}\right) \cdot c \cdot \text{lr}_{\text{eff}}, \quad \text{gap} = k - h_{\text{eff}}, \quad c \in \{1.0, 0.3\}$$

so failing at something challenging still teaches. Effort accumulates as $e \leftarrow 0.95e + \ell/20$ with ℓ the response latency, normalized by $E_{\text{ref}}=3.0$. Fatigue acts on two channels:

$$h_{\text{eff}} = \max(0.5, h - \phi e_n^{1.5}), \quad \text{lr}_{\text{eff}} = \text{lr} \cdot \max(0.15, 1 - 0.7\phi e_n^{1.5})$$

The second is essential: without it, fatigue merely widens the difficulty gap and therefore moves the agent *toward* the bell curve’s peak, so a fatigued student would learn *more* per session — an artefact that renders any fatigue sweep uninterpretable. Rest multiplies effort by 0.7, or 0.93 once e_n exceeds 1.3 (hysteresis), and costs 0.01 of ability. Frustration rises 0.25 per error, 0.3 more when failing at a mastered level, falls 0.35 on accessible successes, halves on rest, and triggers absorbing dropout at $0.95\times$ threshold for three consecutive sessions. Profiles draw $h_0 \in [1.5, 3.5]$, $\text{lr} \in [0.7, 1.3]$, threshold $\in [1.8, 3.0]$.

B Agent specification

Configuration: $\lambda_p=0.15$, $\lambda_c=0.02$, $\theta=0.7$, $\alpha_p=\alpha_c=0.3$, $w_{cp}=-0.05$, $w_{pc}=0.05$, $\eta=0.3$, $\beta=0.2$.

```
def select(self, rng, env):
    dp = self.d_prog - theta          # progress deviation
    dc = self.d_conf - theta          # confidence deviation
    fn = self.frust / self.profile["umbral"] # normalized frustration
    strain = max(0.0, fn - 0.7) / 0.3   # rest gate

    if rest_mode == "gated":           # default arm
        l_rest = 2*max(0.0, dc)*strain + 4*strain
    elif rest_mode == "none":          # viability ablation
        l_rest = -1e9
    else:                              # drive-driven ablation
        l_rest = 3.0*max(0.0, dc) + 2.0*strain - 1.2

    logits = [c0*dp + c1*dc,          # UP
              c2*(abs(dp) + abs(dc)) + c3, # STAY
              c4*dc + c5*dp,          # DOWN
              l_rest]                 # REST
    return ACTIONS[argmax(logits) if action_mode=="argmax"
                  else sample_softmax(logits, rng)]
```

In the default (gated) mode every term of l_{rest} is multiplied by strain , so the drives do not participate in the rest decision: along the viability dimension the agent is as reactive as the baselines it is compared against. This is why the viability claim of Section 7.2 is restricted to the `restdrive` arm.

C Quality gate across the sweep

Table 1: Gate criteria at each sweep point. **Key take-away:** the environment stops discriminating at $\phi=1.6$, where the level-1 trap dissolves.

ϕ	always1	always5 drop	random	oracle	margin
0.0	+0.010	100%	+0.596	+1.139	+0.544 (PASS)
0.3	+0.035	100%	+0.307	+0.868	+0.561 (PASS)
0.6	+0.073	100%	+0.182	+0.533	+0.351 (PASS)
0.9	+0.108	100%	+0.133	+0.406	+0.273 (PASS)
1.2	+0.144	100%	+0.107	+0.357	+0.250 (PASS)
1.6	+0.162	100%	+0.088	+0.316	+0.228 (FAIL)

Only the first criterion fails at $\phi=1.6$ (+0.162 against a 0.15 cutoff); the oracle margin remains healthy.

Sensitivity to the exclusion. At $\phi=1.2$, Γ_{argmax} minus estimator is +0.045, CI [+0.028, +0.062], $d_z=+0.52$; at the excluded $\phi=1.6$ it is +0.089, CI [+0.078, +0.101], $d_z=+1.54$. Including the point strengthens the claim, so the exclusion is conservative.

D Full main sweep (all ten arms)

Table 2: Learning gain / % dropout for every arm at every valid sweep point.

Arm	$\phi=0.0$	$\phi=0.3$	$\phi=0.6$	$\phi=0.9$	$\phi=1.2$
Γ_{learned}	+0.248/0%	+0.148/0%	+0.138/0%	+0.146/1%	+0.143/1%
Γ_{argmax}	+0.256/0%	+0.184/0%	+0.179/0%	+0.199/0%	+0.201/0%
$\Gamma_{\text{restdrive}}$	+0.363/4%	+0.186/7%	+0.165/12%	+0.161/15%	+0.154/16%
Γ_{fixed}	+0.271/0%	+0.142/0%	+0.148/0%	+0.151/0%	+0.147/0%
Γ_{naive}	+0.269/0%	+0.164/0%	+0.139/0%	+0.150/0%	+0.147/0%
Γ_{oracle}	+0.335/0%	+0.149/0%	+0.151/0%	+0.150/0%	+0.145/0%
rule	+0.591/0%	+0.413/0%	+0.270/0%	+0.220/0%	+0.199/0%
rule_norest	+0.595/0%	+0.417/1%	+0.278/13%	+0.229/38%	+0.207/55%
estimator	+1.132/0%	+0.700/0%	+0.360/0%	+0.228/0%	+0.156/0%
estimator_norest	+1.108/16%	+0.612/63%	+0.317/94%	+0.207/100%	+0.156/100%

E Additive decomposition

Table 3: Components of the $\phi=0$ gap on learning gain and time-in-band, against both baselines. **Key take-away:** the ordering of the terms is identical on both metrics and against both comparators.

Component	gain	%	time-in-band	%
Signal quality (oracle — constant, hand policy)	+0.063	7.4	+0.050	8.7
Policy coefficients (re-tuned — hand, constant g)	+0.238	27.7	+0.169	29.4
Interaction	−0.047	−5.5	−0.056	−9.7
Residual (estimator — best Γ)	+0.606	70.5	+0.414	71.7
Total gap vs. model-based estimator	+0.860	100.0	+0.577	100.0
<i>against the model-free estimator (total +0.580): 10.9%, 41.0%, −8.1%, 56.0%</i>				

F Signal-quality contrast across adversity

Table 4: P5: oracle versus constant g , paired over 100 profiles.

ϕ	oracle	constant	diff	95% CI	p	d_z
0.0	+0.335	+0.271	+0.063	[+0.055, +0.071]	6.4e-29	+1.59
0.3	+0.149	+0.142	+0.007	[+0.003, +0.011]	6.0e-04	+0.35
0.6	+0.151	+0.148	+0.003	[+0.000, +0.006]	2.0e-02	+0.24
0.9	+0.150	+0.151	−0.001	[−0.004, +0.001]	2.7e-01	−0.11
1.2	+0.145	+0.147	−0.002	[−0.004, −0.000]	3.2e-02	−0.22

Table 5: TOST equivalence p -values for oracle minus constant across candidate margins. **Key take-away:** from $\phi=0.3$ onward the difference is inside every margin tested, so the conclusion does not depend on δ .

ϕ	diff	$\delta=0.02$	$\delta=0.03$	$\delta=0.05$	$\delta=0.075$	$\delta=0.10$
0.0	+0.063	1.000	1.000	0.999	0.002	<.001
0.3	+0.007	<.001	<.001	<.001	<.001	<.001
0.6	+0.003	<.001	<.001	<.001	<.001	<.001
0.9	-0.001	<.001	<.001	<.001	<.001	<.001
1.2	-0.002	<.001	<.001	<.001	<.001	<.001

479 G Policy and crossover contrasts

Table 6: Paired contrasts over 100 profiles. Left: argmax minus softmax (P4). Right: Γ_{argmax} minus the model-based estimator (P3).

ϕ	P4: argmax – softmax			P3: Γ_{argmax} – estimator		
	diff	95% CI	d_z	diff	95% CI	d_z
0.0	+0.008	[+0.000, +0.016]	+0.21	-0.876	[-0.908, -0.843]	-5.29
0.3	+0.036	[+0.029, +0.043]	+1.00	-0.517	[-0.562, -0.472]	-2.27
0.6	+0.041	[+0.035, +0.048]	+1.25	-0.181	[-0.216, -0.146]	-1.02
0.9	+0.053	[+0.048, +0.058]	+1.98	-0.029	[-0.054, -0.004]	-0.23
1.2	+0.058	[+0.054, +0.061]	+3.23	+0.045	[+0.028, +0.062]	+0.52

480 H Objective-model ablation

Table 7: Learning gain with and without the objective model. **Key take-away:** removing privileged knowledge of the learning curve costs the estimator in the benign regime but *helps* it under adversity — and it still beats the homeostatic agent everywhere.

ϕ	estimator	estimator, no objective model	Γ_{learned}
0.0	+1.132		+0.851
0.6	+0.360		+0.474
1.2	+0.156		+0.275

481 I Viability ablation

Table 8: Dropout % with and without a rest mechanism on *both* sides. **Key take-away:** without rest both architectures collapse, the regulated one slightly later; with rest both are safe. Viability tracks the presence of a rest mechanism, not of homeostatic drives.

ϕ	no rest mechanism		with rest mechanism	
	Γ_{learned}	estimator	Γ_{learned}	estimator
0.0	6.4%	16.4%	0.0%	0.0%
0.6	81.5%	94.5%	0.4%	0.0%
1.2	93.5%	100.0%	1.0%	0.0%

482 J Maintenance-objective experiment

483 The primary task has a terminal objective: accumulate ability by session 40. Gain can be banked,
 484 so more is always better. To test whether homeostatic regulation fares better under a *maintenance*

objective we replace it with **mean readiness** — $\sigma(1.5(h_{\text{eff}} - 3))$, the score the student would obtain if the exam were held that session, averaged over the horizon, with zero credit for every session after quitting. Readiness cannot be banked: fatigue costs continuously rather than only at a deadline.

Table 9: Mean readiness over 60 profiles $\times$ 6 seeds. **Key take-away:** the ordering reverses only under the conjunction of a non-bankable objective, a long horizon, and severe adversity — at the paper’s own setting (top row) the estimators still win.

ϕ	horizon	Γ_{learned}	Γ_{argmax}	estimator	estimator (no obj.)	rule
0.6	40	0.158	0.141	0.203	0.231	0.178
0.6	100	0.110	0.103	0.122	0.168	0.115
0.6	200	0.119	0.134	0.102	0.160	0.097
0.6	400	0.145	0.196	0.120	0.210	0.122
1.2	40	0.099	0.084	0.102	0.114	0.098
1.2	100	0.054	0.049	0.055	0.061	0.053
1.2	200	0.038	0.040	0.039	0.043	0.039
1.2	400	0.041	0.080	0.035	0.049	0.037

Paired contrasts for Γ_{argmax} , $n=60$ profiles: at $\phi=0.6$, $H=40$ it trails the estimator by -0.062 ($d_z=-1.07$) and the model-free estimator by -0.090 ($d_z=-1.29$). At $\phi=0.6$, $H=400$ it leads the estimator by $+0.076$ ($d_z=+2.09$) but trails the model-free one by -0.014 . At $\phi=1.2$, $H=400$ it leads both, by $+0.044$ ($d_z=+0.73$) and $+0.031$ ($d_z=+0.61$), all $p < 10^{-4}$.

A failed first attempt. We first tested the hypothesis by extending the horizon alone while keeping cumulative gain as the objective. That test failed: no baseline ever quit, the estimator’s advantage grew with horizon, and Γ ’s survival fell to 85% at 400 sessions. Lengthening a horizon does not make an accumulative objective a maintenance objective, which is the distinction the sixth condition turns on. We report the failed version because it is what identified the right variable.

K Matched-tuning results

Table 10: Held-out validation gain across five independent coefficient searches per signal. **Key take-away:** the ordering is monotone in signal quality once search variance is accounted for — the earlier non-monotonic result was a single-search artefact — but the spread is small.

Signal	mean (sd) over 5 searches	larger budget (1200)	vs. constant
constant g	+0.472 (0.051)	+0.411	—
learned g	+0.480 (0.068)	+0.545	indistinguishable
oracle g	+0.519 (0.052)	+0.656	higher in 5/5 searches
<i>estimator: +1.132; estimator without objective model: +0.851</i>			

L Policy coefficient search

The four action logits are linear in d_p, d_c with six free coefficients. The original values $(4, -3, -3, 1.5, 4, -3)$ were chosen by hand. To test whether performance is limited by these particular numbers we ran a random search over 250 candidates sampled from $c_0, c_4 \sim U(0, 10)$; $c_1, c_5 \sim U(-10, 2)$; $c_2 \sim U(-10, 0)$; $c_3 \sim U(-2, 5)$, scored on a 30-profile subset and validated on 100 profiles. The matched protocol of Section 6.3 repeats this independently per signal with five search seeds and a strictly disjoint 50-profile validation set; those held-out numbers, not the ones below, support the paper’s conclusions.

Table 11: Effect of coefficient re-tuning at $\phi=0$, shared search, validated on 100 profiles including the 30 used for scoring (partly in-sample; see Section 6.3 for the held-out version).

Arm	hand-chosen	re-tuned	Δ
Γ_{oracle}	+0.335	+0.526	+0.191
Γ_{fixed}	+0.271	+0.509	+0.238
Γ_{learned}	+0.248	+0.266	+0.018
<i>estimator: +1.132</i>			

506 M Shape-shift results and quality gate

Table 12: Peak of the learning curve moved at session 20; $\phi=0.6$; 100 paired profiles. **Key take-away:** all four points pass the gate, and the estimator’s lead grows as its own model is corrupted.

peak	Γ_{argmax}	Γ_{oracle}	estimator	gap	d_z	gate margin
0.5	+0.179	+0.151	+0.360	−0.181	−1.02	+0.370 (PASS)
1.0	+0.145	+0.110	+0.345	−0.200	−1.19	+0.316 (PASS)
1.5	+0.114	+0.068	+0.324	−0.210	−1.32	+0.294 (PASS)
2.0	+0.103	+0.057	+0.301	−0.198	−1.26	+0.280 (PASS)

507 N Complete sweep metrics

Table 13: Time-in-band, conditional on study sessions.

Arm	$\phi=0.0$	$\phi=0.3$	$\phi=0.6$	$\phi=0.9$	$\phi=1.2$
Γ_{learned}	0.271	0.350	0.519	0.610	0.644
Γ_{argmax}	0.252	0.340	0.541	0.670	0.715
$\Gamma_{\text{restdrive}}$	0.289	0.343	0.519	0.578	0.597
Γ_{fixed}	0.229	0.354	0.571	0.637	0.656
Γ_{naive}	0.250	0.341	0.498	0.610	0.648
Γ_{oracle}	0.279	0.400	0.599	0.647	0.660
rule	0.364	0.409	0.498	0.612	0.670
rule_norest	0.365	0.414	0.510	0.613	0.662
estimator	0.805	0.780	0.592	0.459	0.381
estimator_norest	0.796	0.749	0.589	0.447	0.371

Table 14: Exam score on effective ability. **Key take-away:** the ordering matches learning gain.

Arm	$\phi=0.0$	$\phi=0.3$	$\phi=0.6$	$\phi=0.9$	$\phi=1.2$
Γ_{learned}	0.446	0.205	0.085	0.041	0.031
Γ_{argmax}	0.448	0.192	0.072	0.038	0.028
$\Gamma_{\text{restdrive}}$	0.484	0.169	0.055	0.028	0.024
Γ_{fixed}	0.454	0.164	0.047	0.028	0.023
Γ_{naive}	0.454	0.189	0.071	0.033	0.027
Γ_{oracle}	0.474	0.150	0.039	0.025	0.023
rule	0.556	0.251	0.087	0.039	0.026
rule_norest	0.557	0.250	0.083	0.035	0.024
estimator	0.716	0.338	0.084	0.034	0.024
estimator_norest	0.708	0.290	0.073	0.031	0.025

Table 15: Mean REST sessions out of 40. **Key take-away:** the regulated arms spend much of the budget resting — the price of the viability they buy.

Arm	$\phi=0.0$	$\phi=0.3$	$\phi=0.6$	$\phi=0.9$	$\phi=1.2$
Γ_{learned}	9.1	8.0	6.7	6.2	6.0
Γ_{argmax}	8.8	6.5	4.5	3.4	3.0
$\Gamma_{\text{restdrive}}$	5.0	4.7	4.4	4.2	4.2
Γ_{fixed}	6.7	5.7	5.0	5.0	5.0
Γ_{naive}	7.3	6.7	5.9	5.6	5.6
Γ_{oracle}	6.3	5.7	5.1	5.1	5.1
rule	0.1	0.3	0.8	1.5	2.0
rule_norest	0.0	0.0	0.0	0.0	0.0
estimator	1.1	2.4	4.0	5.0	5.4
estimator_norest	0.0	0.0	0.0	0.0	0.0

Table 16: Satiation-signal validity. **Key take-away:** validity degrades with adversity for the learned arm but not the oracle.

Arm	$\phi=0.0$	$\phi=0.3$	$\phi=0.6$	$\phi=0.9$	$\phi=1.2$
Γ_{learned}	+0.493	+0.415	+0.351	+0.303	+0.281
Γ_{argmax}	+0.550	+0.443	+0.505	+0.456	+0.452
$\Gamma_{\text{restdrive}}$	+0.441	+0.488	+0.470	+0.431	+0.397
Γ_{fixed}	—	—	—	—	—
Γ_{naive}	—	—	—	—	—
Γ_{oracle}	+0.994	+0.861	+0.896	+0.920	+0.909
rule	—	—	—	—	—
rule_norest	—	—	—	—	—
estimator	—	—	—	—	—
estimator_norest	—	—	—	—	—

508 O Full sweep figure

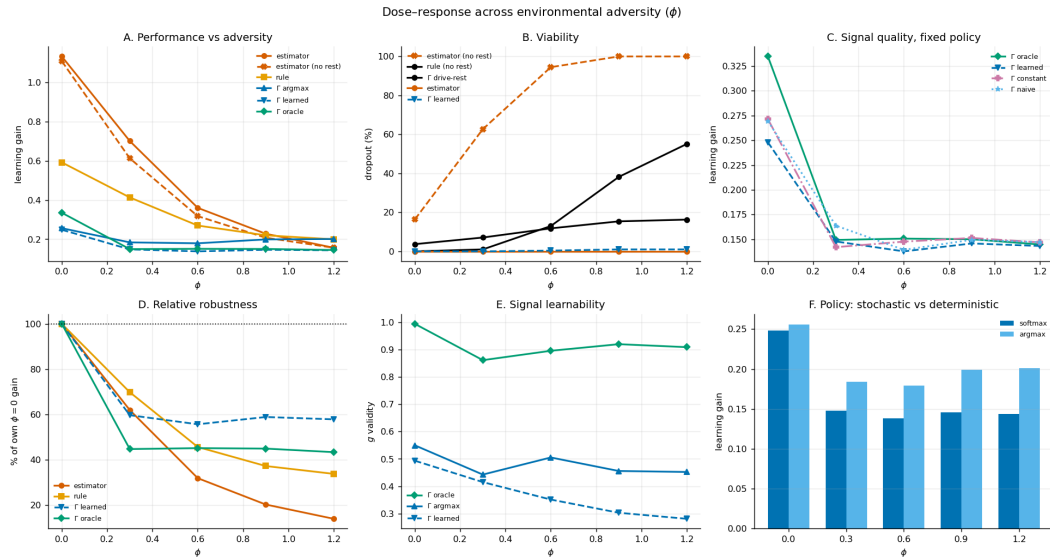


Figure 1: Six-panel summary, Okabe-Ito colourblind-safe palette with distinct markers and line styles so the panels remain legible in greyscale. **A:** performance across adversity. **B:** viability. **C:** signal quality under the fixed hand policy. **D:** robustness normalized to each arm's own $\phi=0$ value. **E:** signal learnability. **F:** stochastic versus deterministic policy.

P Compute and LLM usage

All experiments run on a commodity laptop with no GPU; the main sweep of 60,000 episodes takes 31.4 s single-threaded and a single decision costs 14 μ s (full specifications in Appendix Q).

We disclose LLM usage for transparency, although it is not a component of the core method: no language model participates in the agent, the environment, the baselines, or the analysis, all of which are explicit numerical simulations. LLMs served as engineering assistants in two capacities — code assistance, via Devin as an IDE together with Kimi 3 and DeepSeek v4 Pro, and Claude Opus 5 and Claude Fable 5; and writing and editing assistance, via Claude Opus 5 and Claude Fable 5. All experimental design decisions, the committed predictions, the statistical procedures, and every interpretation are the authors’ own, and all reported numbers were produced by executing the released notebook.

Q Compute resources

Intel Core i7-1165G7, 4 cores, 32 GB RAM, **no GPU**, pure Python with `scipy` and `matplotlib` as the only non-standard dependencies. The main sweep (60,000 episodes across 6 adversity points and 10 arms) takes 31.4 s single-threaded; the multi-seed coefficient search adds roughly 5 minutes; the maintenance-objective experiment about 4 minutes. Per-episode cost is 0.55 ms for 40 decisions, i.e. 14 μ s per decision, consistent with the operator’s $O(n^2 + A)$ per-session arithmetic and absence of gradients, value functions, or replay. The entire project consumed less compute than a single GPU-minute.

R Reproducibility

Ten seeds are derived from master seed 20260820 via SHA-256, each mapped to the next prime above $10007 + (\text{digest mod } 900000)$; profiles come from a separate fixed seed. The released notebook reproduces every reported number and asserts the prediction hash before running. Dependencies are `scipy` and `matplotlib`.

S Preregistered prediction text (verbatim)

The authoritative artifact is `predictions.txt` in the supplement, whose SHA-256 digest is 6e888c7e46e1dd306b6adc601a45cc08ff914a24d2f474a913a1ff2907bfc084. It contains one prediction per line in sorted key order, joined with newlines. Below the same text is shown with display-only wrapping; continuation lines are indented and are *not* part of the hashed string.

```
P1: bayesian_norest (estimacion sin gestion de viabilidad) colapsa al subir
    phi: dropout alto, gain al piso.
P2: Los brazos Gamma se degradan proporcionalmente menos que el bayesiano a
    lo largo del barrido de phi.
P3: Existe phi* dentro del rango valido de la puerta de calidad donde algun
    brazo Gamma supera al bayesiano en gain.
P4: gamma_learned_argmax supera a gamma_learned en todos los puntos validos
    del barrido.
P5: Si la calidad de g se transmite a la conducta, gamma_oracle se separa de
    gamma_no_learning en algun punto del barrido.
P6: g_validity de gamma_learned decrece monotonamente al subir phi.
P7: Bajo desplazamiento del PICO de la campana de aprendizaje (cambio de
    forma, no de escala), los brazos Gamma ganan terreno relativo frente al
    bayesiano, que queda mal especificado y no puede absorberlo re-estimando h.
```

T Source code

Self-contained: no external dataset, no pretrained model, only `scipy` and `matplotlib` beyond the standard library. **Naming note:** the code predates the paper’s terminology — `BayesianAgent` is the paper’s *estimator*, `BayesianNoObjAgent` the estimator without an objective model, `ReglaAgent` the *rule*, `gamma_no_learning` the *constant-g* arm; `nivel`, `ganancia` and `descansar` are *level*, *gain* and *rest*.

558 T.1 Seeds and determinism

```

559 import hashlib, math, random
560 from collections import defaultdict
561
562 NIVELES=[1,2,3,4,5]
563 PRESUPUESTO=40
564 E_REF=3.0
565
566 def _is_prime(n):
567     if n<2: return False
568     if n in (2,3,5,7): return True
569     if n%2==0 or n%3==0: return False
570     i=5
571     while i*i<=n:
572         if n%i==0 or n%(i+2)==0: return False
573         i+=6
574     return True
575 def _next_prime(n):
576     if n<=2: return 2
577     c=n|1
578     while not _is_prime(c): c+=2
579     return c
580 def derive_seeds(master,n=10):
581     s=[]
582     for i in range(n):
583         d=hashlib.sha256(f"{master}:run_{i}".encode()).hexdigest()
584         s.append(_next_prime(10007+int(d,16)%900000))
585     return s
586
587 def rasch_p(h,nivel): return 1.0/(1.0+math.exp(-1.5*(h-nivel)))
588 PEAK_MODEL=0.5 # peak assumed by any agent that models the curve (misspecifiable)
589 def bell_curve(gap,peak=PEAK_MODEL): return 0.08*math.exp(-((gap-peak)**2)/0.5)

```

590 T.2 Environment

```

591 class StudyEnv:
592     """v4: fatiga normalizada + histeresis + shift de regimen + examen en h_eff."""
593     def __init__(self, h0, lr_mult=1.0, phi=0.0, lambda_olv=0.01,
594                 hysteresis=False, e_crit=1.3, shift_mult=1.0, shift_at=20,
595                 peak_gap=PEAK_MODEL, peak_new=None, peak_at=20):
596         self.h=h0; self.h0=h0; self.lr_mult=lr_mult
597         self.phi=phi; self.phi0=phi
598         self.lambda_olv=lambda_olv
599         self.hysteresis=hysteresis; self.e_crit=e_crit
600         self.shift_mult=shift_mult; self.shift_at=shift_at
601         self.peak_gap=peak_gap; self.peak_new=peak_new; self.peak_at=peak_at
602         self.esfuerzo=0.0
603
604     def tick(self,s):
605         if self.shift_mult!=1.0 and s==self.shift_at:
606             self.phi=self.phi0*self.shift_mult
607         if self.peak_new is not None and s==self.peak_at:
608             self.peak_gap=self.peak_new
609
610     @property
611     def esf_n(self): return self.esfuerzo/E_REF
612     @property
613     def h_eff(self): return max(0.5, self.h - self.phi*self.esf_n**1.5)
614     @property
615     def lr_eff(self):
616         # la fatiga degrada la CAPACIDAD de aprender, no solo la dificultad aparente
617         return self.lr_mult*max(0.15, 1.0 - 0.7*self.phi*self.esf_n**1.5)
618
619     def expected_gain(self,nivel):
620         gap=nivel-self.h_eff; p=rasch_p(self.h_eff,nivel)
621         return bell_curve(gap,self.peak_gap)*(p*1.0+(1-p)*0.3)*self.lr_eff
622     def in_zpd(self,nivel):

```

```

623         g=[self.expected_gain(k) for k in NIVELES]; mx=max(g)
624         return (self.expected_gain(nivel)>=0.5*mx) if mx>0 else False
625
626     def exercise(self,nivel,rng):
627         p=rasch_p(self.h_eff,nivel)
628         correcto= rng.random()<p
629         gap=max(0.0,nivel-self.h_eff)
630         latencia=rng.lognormvariate(math.log(5)+0.6*gap,0.4)
631         self.esfuerzo=0.95*self.esfuerzo+latencia/20.0
632         gain=bell_curve(nivel-self.h_eff,self.peak_gap)*(1.0 if correcto else 0.3)*self.lr_eff
633         self.h=min(6.0,self.h+gain)
634         return {"correcto":correcto,"latencia":round(latencia,2),"gain":gain}
635
636     def descansar(self):
637         if self.hysteresis and self.esf_n>self.e_crit: self.esfuerzo*=0.93
638         else: self.esfuerzo*=0.7
639         self.h=max(0.5,self.h-self.lambda_olv)

```

640 T.3 Homeostatic agent

641 The coupling uses the *receiving* drive's pressure, matching Eq. (1); the legacy sender-modulated branch is kept for reference
642 only and used for no reported result.

```

643 class GCfg:
644     def __init__(self,**kw):
645         self.lam_p=kw.get("lam_p",0.15); self.lam_c=kw.get("lam_c",0.02)
646         self.theta=kw.get("theta",0.7)
647         self.alpha_p=kw.get("alpha_p",0.3); self.alpha_c=kw.get("alpha_c",0.3)
648         self.w_cp=kw.get("w_cp",-0.05); self.w_pc=kw.get("w_pc",0.05)
649         self.g_mode=kw.get("g_mode","learned")
650         self.n_drives=kw.get("n_drives",2); self.coupling=kw.get("coupling",True)
651         self.coupling_mode=kw.get("coupling_mode",
652             "receiver") # receiver (Eq.1 / design intent) | sender (legacy bug)
653         self.eta=kw.get("eta",0.3); self.beta=kw.get("beta",0.2)
654         self.g_init_p=kw.get("g_init_p",0.3); self.g_init_c=kw.get("g_init_c",0.5)
655         self.g_naive=kw.get("g_naive",0.5)
656         self.action_mode=kw.get("action_mode","softmax") # softmax | argmax
657         self.rest_mode=kw.get("rest_mode","gated") # gated | drive | none
658         self.coef=kw.get("coef",(4.0,-3.0,-3.0,1.5,4.0,-3.0)) # up_p,up_c,stay_k,stay_b,down_c,
659             down_p # gated | drive
660
661 class GammaAgent:
662     def __init__(self,cfg,profile):
663         self.cfg=cfg; self.p=profile
664         self.d_prog=0.5; self.d_conf=0.5
665         self.g_prog={k:cfg.g_init_p for k in NIVELES}
666         self.g_conf={k:cfg.g_init_c for k in NIVELES}
667         self.acc_ema={}; self.nivel=1; self.frust=0.0
668         self.consec_high=0; self.abandono=False; self.dropout_session=None
669     def _pressure(self,d): return min(1.0,abs(d-self.cfg.theta)/0.8)
670     def basal(self):
671         m=self.acc_ema.get(self.nivel,0.5)
672         self.d_prog=min(2.0,self.d_prog+self.cfg.lam_p*m)
673         if self.cfg.n_drives>=2:
674             self.d_conf=min(2.0,self.d_conf+self.cfg.lam_c)
675             if self.cfg.coupling:
676                 pc=self._pressure(self.d_conf); pp=self._pressure(self.d_prog)
677                 if self.cfg.coupling_mode=="receiver":
678                     # Eq.(1): modulated by the RECEIVING drive's own pressure
679                     m_to_prog, m_to_conf = pp, pc
680                 else:
681                     m_to_prog, m_to_conf = pc, pp # legacy: sender's pressure
682             if self.d_conf>self.cfg.theta+0.05:
683                 self.d_prog=max(0.0,self.d_prog+self.cfg.w_cp*m_to_prog)
684             if self.d_prog>self.cfg.theta+0.05:
685                 self.d_conf=max(0.0,self.d_conf+self.cfg.w_pc*m_to_conf)
686     def select(self,rng,env):
687         th=self.cfg.theta

```

```

688         dp=self.d_prog-th
689         dc=(self.d_conf-th) if self.cfg.n_drives>=2 else 0.0
690         fn=self.frust/max(self.p["umbral"],0.1)
691         strain=max(0.0,fn-0.7)/0.3
692         if self.cfg.n_drives>=2:
693             if self.cfg.rest_mode=="gated":
694                 l_rest=2*max(0.0,dc)*strain+4*strain
695             elif self.cfg.rest_mode=="none":
696                 l_rest=-1e9 # rest unavailable: isolates drives from the frustration
697             gate
698         else:
699             l_rest=3.0*max(0.0,dc)+2.0*strain-1.2
700             c=self.cfg.coef
701             logits=[c[0]*dp+c[1]*dc, c[2]*(abs(dp)+abs(dc))+c[3], c[4]*dc+c[5]*dp, l_rest]
702         else:
703             logits=[4*dp,-3*abs(dp)+1.5,-4*dp,4*strain]
704             acts=["subir","mantener","bajar","descansar"]
705             if self.cfg.action_mode=="argmax":
706                 return acts[max(range(4),key=lambda i:logits[i])]
707             mx=max(logits); exps=[math.exp(min(50,1-mx)) for l in logits]
708             tot=sum(exps); r=rng.random()*tot; cum=0.0
709             for i,e in enumerate(exps):
710                 cum+=e
711                 if r<cum: return acts[i]
712             return "mantener"
713     def study(self,env,outcome,nivel):
714         c=outcome["correcto"]; eb=self.acc_ema.get(nivel,0.5)
715         self.acc_ema[nivel]=(1-self.cfg.beta)*eb+self.cfg.beta*float(c)
716         if self.cfg.g_mode=="learned":
717             a=self.acc_ema[nivel]
718             self.g_prog[nivel]=(1-self.cfg.eta)*self.g_prog[nivel]+self.cfg.eta*(4.0*a*(1.0-a))
719             self.g_conf[nivel]=(1-self.cfg.eta)*self.g_conf[nivel]+self.cfg.eta*a
720         elif self.cfg.g_mode=="oracle":
721             self.g_prog[nivel]=min(1.0,env.expected_gain(nivel)/0.042)
722             self.g_conf[nivel]=rasch_p(env.h_eff,nivel)
723         if self.cfg.g_mode=="naive": gp=gc=self.cfg.g_naive
724         else: gp=self.g_prog[nivel]; gc=self.g_conf[nivel]
725         if c:
726             self.d_prog=max(0.0,self.d_prog-self.cfg.alpha_p*gp)
727             if self.cfg.n_drives>=2:
728                 self.d_conf=max(0.0,self.d_conf-self.cfg.alpha_c*gc)
729             if outcome["latencia"]<15: self.frust=max(0.0,self.frust-0.35)
730         else:
731             if self.cfg.n_drives>=2: self.d_conf=min(2.0,self.d_conf+0.1)
732             self.frust+=0.25
733             if eb>=0.7: self.frust+=0.3

```

734 T.4 Baselines

```

735 class ReglaAgent:
736     def __init__(self,profile,rest=True):
737         self.p=profile; self.rest=rest; self.nivel=1; self.frust=0.0
738         self.consec_high=0; self.abandono=False; self.dropout_session=None
739         self.streak_c=0; self.streak_e=0; self.acc_ema={}
740     def select(self,rng,env):
741         if self.rest and self.frust>0.6*self.p["umbral"]: return "descansar"
742         if self.streak_e>=2: return "bajar"
743         if self.streak_c>=3: return "subir"
744         return "mantener"
745     def study(self,env,outcome,nivel):
746         if outcome["correcto"]: self.streak_c+=1; self.streak_e=0
747         else: self.streak_e+=1; self.streak_c=0
748
749 class BayesianNoObjAgent:
750     """State estimation WITHOUT an objective model: tracks h_hat from outcomes and
751     simply matches the level to estimated ability. Isolates estimation from
752     privileged knowledge of where the learning curve peaks."""
753     def __init__(self,profile,rest=True):

```

```

754         self.p=profile; self.rest=rest; self.nivel=1; self.h_hat=2.5
755         self.frust=0.0; self.consec_high=0; self.abandono=False
756         self.dropout_session=None; self.acc_ema={}
757     def select(self, rng, env):
758         if self.rest and self.frust>0.6*self.p["umbral"]: return "descansar"
759         best=min(NIVELES, key=lambda k: abs(k-self.h_hat))
760         if best>self.nivel: return "subir"
761         if best<self.nivel: return "bajar"
762         return "mantener"
763     def study(self, env, outcome, nivel):
764         p=rasch_p(self.h_hat, nivel)
765         self.h_hat+=0.3*(float(outcome["correcto"])-p)
766         self.h_hat=max(0.5, min(6.0, self.h_hat))
767
768 class BayesianAgent:
769     def __init__(self, profile, rest=True):
770         self.p=profile; self.rest=rest; self.nivel=1; self.h_hat=2.5
771         self.frust=0.0; self.consec_high=0; self.abandono=False
772         self.dropout_session=None; self.acc_ema={}
773     def select(self, rng, env):
774         if self.rest and self.frust>0.6*self.p["umbral"]: return "descansar"
775         best, bg=self.nivel, -1
776         for k in NIVELES:
777             gap=k-self.h_hat; p=rasch_p(self.h_hat, k)
778             g=bell_curve(gap)*(p+(1-p)*0.3)
779             if g>bg: best, bg=k, g
780         if best>self.nivel: return "subir"
781         if best<self.nivel: return "bajar"
782         return "mantener"
783     def study(self, env, outcome, nivel):
784         p=rasch_p(self.h_hat, nivel)
785         self.h_hat+=0.3*(float(outcome["correcto"])-p)
786         self.h_hat=max(0.5, min(6.0, self.h_hat))
787
788 class FixedAgent:
789     def __init__(self, profile, mode):
790         self.p=profile; self.mode=mode; self.nivel=1; self.frust=0.0
791         self.consec_high=0; self.abandono=False; self.dropout_session=None
792         self.acc_ema={}
793     def select(self, rng, env):
794         if self.mode=="always1": t=1
795         elif self.mode=="always5": t=5
796         elif self.mode=="random": t=rng.choice(NIVELES)
797         else: t=max(NIVELES, key=lambda k: env.expected_gain(k))
798         if t>self.nivel: return "subir"
799         if t<self.nivel: return "bajar"
800         return "mantener"
801     def study(self, env, outcome, nivel): pass
802
803 def shared_frust_update(agent, outcome, eb):
804     if outcome["correcto"]:
805         if outcome["latencia"]<15: agent.frust=max(0.0, agent.frust-0.35)
806     else:
807         agent.frust+=0.25
808     if eb>=0.7: agent.frust+=0.3

```

809 T.5 Profiles and episode runner

```

810 def gen_profiles(n=100, seed=2026):
811     rng=random.Random(seed)
812     return [{"h0":round(rng.uniform(1.5, 3.5), 2),
813             "lr_mult":round(rng.uniform(0.7, 1.3), 2),
814             "umbral":round(rng.uniform(1.8, 3.0), 1)} for _ in range(n)]
815
816 def run_episode(agent, profile, seed, is_gamma, env_kw, log=False):
817     env=StudyEnv(profile["h0"], lr_mult=profile["lr_mult"], **env_kw)
818     rng=random.Random(seed); traj=[]; zpd=0; nst=0; ndesc=0; s=0
819     for s in range(PRESUPUESTO):

```

```

820         if agent.abandono: break
821         env.tick(s)
822         if is_gamma: agent.basal()
823         a=agent.select(rng,env)
824         if a=="subir": agent.nivel=min(5,agent.nivel+1)
825         elif a=="bajar": agent.nivel=max(1,agent.nivel-1)
826         out=None; inz=False
827         if a=="descansar":
828             env.descansar(); agent.frust*=0.5; ndesc+=1
829         else:
830             inz=env.in_zpd(agent.nivel)
831             out=env.exercise(agent.nivel,rng); nst+=1
832             if inz: zpd+=1
833             if is_gamma: agent.study(env,out,agent.nivel)
834             else:
835                 eb=agent.acc_ema.get(agent.nivel,0.5)
836                 agent.acc_ema[agent.nivel]=0.8*eb+0.2*float(out["correcto"])
837                 shared_frust_update(agent,out,eb)
838                 agent.study(env,out,agent.nivel)
839             if agent.frust>=0.95*profile["umbral"]: agent.consec_high+=1
840             else: agent.consec_high=0
841             if agent.consec_high>=3 and not agent.abandono:
842                 agent.abandono=True; agent.dropout_session=s
843             if log:
844                 traj.append({"s":s,"nivel":agent.nivel,"action":a,
845                     "correcto":out["correcto"] if out else None,
846                     "d_prog":round(getattr(agent,"d_prog",0),3),
847                     "d_conf":round(getattr(agent,"d_conf",0),3),
848                     "frust":round(agent.frust,2),"h":round(env.h,3),
849                     "h_eff":round(env.h_eff,3),"esf_n":round(env.esf_n,3),"in_zpd":inz})
850             m={"ganancia":round(env.h-profile["h0"],3),
851                 "h_final":round(env.h,3),
852                 "exam":round(rasch_p(env.h,3),3),
853                 "exam_taken":round(0.0 if agent.abandono else rasch_p(env.h_eff,3),3),
854                 "exam_eff":round(rasch_p(env.h_eff,3),3),
855                 "esf_n_final":round(env.esf_n,3),
856                 "tiz":round(zpd/max(1,nst),3),
857                 "tib_full":round(zpd/PRESUPUEST0,3),
858                 "dropout":agent.abandono,"dropout_session":agent.dropout_session,
859                 "n_descansar":ndesc,"n_study":nst,"nivel_final":agent.nivel,
860                 "sessions_used":s+1}
861             if is_gamma and agent.cfg.g_mode in ("learned","oracle","fixed","naive"):
862                 gains=[env.expected_gain(k) for k in NIVELES]
863                 gps=[agent.g_prog[k] for k in NIVELES]
864                 mg=sum(gains)/5; mp=sum(gps)/5
865                 cov=sum((gains[i]-mg)*(gps[i]-mp) for i in range(5))
866                 vg=math.sqrt(sum((g-mg)**2 for g in gains)); vp=math.sqrt(sum((g-mp)**2 for g in gps))
867                 m["g_validity"]=round(cov/(vg*vp),3) if vg>0 and vp>0 else None
868                 m["g_prog_final"]={str(k):round(agent.g_prog[k],3) for k in NIVELES}
869                 m["true_gain_final"]={str(k):round(env.expected_gain(k),4) for k in NIVELES}
870             return m,traj
871
872 CFG_BASE=dict(lam_p=0.15,alpha_p=0.3,alpha_c=0.3,theta=0.7)

```

873 U Analysis code: paired inference and equivalence

874 The design is within-subject — the same 100 profiles face every arm — so contrasts are paired and
875 equivalence is tested explicitly rather than inferred from non-significance.

```

876 from scipy.stats import ttest_rel, t as tdist
877
878 def per_profile(arm, phi):
879     # one unit per profile
880     return [mean(run(arm, profile, seed, phi) for seed in SEEDS)
881             for profile in PROFILES]
882
883 def paired_contrast(a, b, phi):

```

```

883     xa, xb = per_profile(a, phi), per_profile(b, phi)
884     d = [u - v for u, v in zip(xa, xb)]
885     n = len(d); m = mean(d); se = stdev(d) / sqrt(n)
886     h = se * tdist.ppf(0.975, n - 1)      # 95% CI half-width
887     t, p = ttest_rel(xa, xb)              # NOT ttest_ind: paired design
888     return m, (m - h, m + h), p, m / stdev(d)
889
890 def tost(d, delta):                      # two one-sided tests
891     n = len(d); m = mean(d); se = stdev(d) / sqrt(n)
892     return max(1 - tdist.cdf((m + delta) / se, n - 1),
893               tdist.cdf((m - delta) / se, n - 1))
894
895 # P3 is an existence claim: max-statistic sign-flip permutation
896 T_obs = max(paired_t(arm, phi) for arm in GAMMA_ARMS for phi in PHIS_VALID)
897 null = [max(paired_t(arm, phi, flip=s) for arm in GAMMA_ARMS
898             for phi in PHIS_VALID) for s in random_sign_flips(2000)]
899 p_P3 = (sum(t >= T_obs for t in null) + 1) / 2001

```

NeurIPS Paper Checklist

1. Claims

Question: Do the main claims made in the abstract and introduction accurately reflect the paper’s contributions and scope?

Answer: [Yes]

Justification: The abstract and Introduction state the split answer (g is learnable; the learned signal does not reach behavior), the viability result, and the RLP decomposition, matching the Results section exactly. The abstract reports the three-way decomposition with effect sizes, states that a learned signal underperforms a constant, reports the refuted prediction P7, and states that the crossover result concerns robustness, and the Limitations section bounds the claim by noting the crossover occurs in a low-gain regime.

Guidelines:

- The answer [N/A] means that the abstract and introduction do not include the claims made in the paper.
- The abstract and/or introduction should clearly state the claims made, including the contributions made in the paper and important assumptions and limitations. A [No] or [N/A] answer to this question will not be perceived well by the reviewers.
- The claims made should match theoretical and experimental results, and reflect how much the results can be expected to generalize to other settings.
- It is fine to include aspirational goals as motivation as long as it is clear that these goals are not attained by the paper.

2. Limitations

Question: Does the paper discuss the limitations of the work performed by the authors?

Answer: [Yes]

Justification: The dedicated Limitations section covers the low-gain regime in which the crossover occurs, incomplete operationalization of signal validity (only g_{prog} , terminal, five points), the functional-form mismatch that caps achievable validity and partially confounds the oracle contrast, the fact that policy tuning was a random search rather than an optimum (making the residual an upper bound), the still-untested applicability conditions (2) and (4), the self-attested commitment hash and the un-committed gate thresholds, and simulator simplicity.

Guidelines:

- The answer [N/A] means that the paper has no limitation while the answer [No] means that the paper has limitations, but those are not discussed in the paper.
- The authors are encouraged to create a separate “Limitations” section in their paper.
- The paper should point out any strong assumptions and how robust the results are to violations of these assumptions (e.g., independence assumptions, noiseless settings, model well-specification, asymptotic approximations only holding locally). The authors should reflect on how these assumptions might be violated in practice and what the implications would be.
- The authors should reflect on the scope of the claims made, e.g., if the approach was only tested on a few datasets or with a few runs. In general, empirical results often depend on implicit assumptions, which should be articulated.
- The authors should reflect on the factors that influence the performance of the approach. For example, a facial recognition algorithm may perform poorly when image resolution is low or images are taken in low lighting. Or a speech-to-text system might not be used reliably to provide closed captions for online lectures because it fails to handle technical jargon.
- The authors should discuss the computational efficiency of the proposed algorithms and how they scale with dataset size.
- If applicable, the authors should discuss possible limitations of their approach to address problems of privacy and fairness.

952 • While the authors might fear that complete honesty about limitations might be used by
953 reviewers as grounds for rejection, a worse outcome might be that reviewers discover
954 limitations that aren't acknowledged in the paper. The authors should use their best
955 judgment and recognize that individual actions in favor of transparency play an impor-
956 tant role in developing norms that preserve the integrity of the community. Reviewers
957 will be specifically instructed to not penalize honesty concerning limitations.

958 3. Theory assumptions and proofs

959 Question: For each theoretical result, does the paper provide the full set of assumptions and
960 a complete (and correct) proof?

961 Answer: [N/A]

962 Justification: The paper presents no theorems or formal proofs. Its contributions are
963 empirical, and the equations given in the Architecture section and Appendix B are model
964 definitions rather than theoretical results requiring proof.

965 Guidelines:

- 966 • The answer [N/A] means that the paper does not include theoretical results.
- 967 • All the theorems, formulas, and proofs in the paper should be numbered and cross-
968 referenced.
- 969 • All assumptions should be clearly stated or referenced in the statement of any theorems.
- 970 • The proofs can either appear in the main paper or the supplemental material, but if
971 they appear in the supplemental material, the authors are encouraged to provide a short
972 proof sketch to provide intuition.
- 973 • Inversely, any informal proof provided in the core of the paper should be complemented
974 by formal proofs provided in appendix or supplemental material.
- 975 • Theorems and Lemmas that the proof relies upon should be properly referenced.

976 4. Experimental result reproducibility

977 Question: Does the paper fully disclose all the information needed to reproduce the main ex-
978 perimental results of the paper to the extent that it affects the main claims and/or conclusions
979 of the paper (regardless of whether the code and data are provided or not)?

980 Answer: [Yes]

981 Justification: Appendix A gives the complete environment specification with all constants;
982 Appendix B gives the agent configuration and the action-selection code; Appendix F doc-
983 uments the seed derivation procedure (SHA-256 from master seed 20260820), the profile
984 generation, and the compute requirements. The Experimental Design section specifies
985 the arm definitions, the profile-clustered statistical protocol, and the episode counts. An
986 anonymized reproducible notebook is included in the supplementary material, and the
987 SHA-256 commitment hash of the preregistered predictions is reported in the paper and
988 recomputable from the verbatim prediction text in the appendix.

989 Guidelines:

- 990 • The answer [N/A] means that the paper does not include experiments.
- 991 • If the paper includes experiments, a [No] answer to this question will not be perceived
992 well by the reviewers: Making the paper reproducible is important, regardless of
993 whether the code and data are provided or not.
- 994 • If the contribution is a dataset and/or model, the authors should describe the steps taken
995 to make their results reproducible or verifiable.
- 996 • Depending on the contribution, reproducibility can be accomplished in various ways.
997 For example, if the contribution is a novel architecture, describing the architecture fully
998 might suffice, or if the contribution is a specific model and empirical evaluation, it may
999 be necessary to either make it possible for others to replicate the model with the same
1000 dataset, or provide access to the model. In general, releasing code and data is often
1001 one good way to accomplish this, but reproducibility can also be provided via detailed
1002 instructions for how to replicate the results, access to a hosted model (e.g., in the case
1003 of a large language model), releasing of a model checkpoint, or other means that are
1004 appropriate to the research performed.

- While NeurIPS does not require releasing code, the conference does require all submissions to provide some reasonable avenue for reproducibility, which may depend on the nature of the contribution. For example
 - (a) If the contribution is primarily a new algorithm, the paper should make it clear how to reproduce that algorithm.
 - (b) If the contribution is primarily a new model architecture, the paper should describe the architecture clearly and fully.
 - (c) If the contribution is a new model (e.g., a large language model), then there should either be a way to access this model for reproducing the results or a way to reproduce the model (e.g., with an open-source dataset or instructions for how to construct the dataset).
 - (d) We recognize that reproducibility may be tricky in some cases, in which case authors are welcome to describe the particular way they provide for reproducibility. In the case of closed-source models, it may be that access to the model is limited in some way (e.g., to registered users), but it should be possible for other researchers to have some path to reproducing or verifying the results.

5. Open access to data and code

Question: Does the paper provide open access to the data and code, with sufficient instructions to faithfully reproduce the main experimental results, as described in supplemental material?

Answer: [Yes]

Justification: An anonymized self-contained notebook reproducing every reported number, table, and figure is included in the supplementary material. No external dataset is required: the environment is a simulator fully specified in Appendix A, and all stochasticity is controlled by the documented seed derivation. Dependencies are limited to `scipy` and `matplotlib`.

Guidelines:

- The answer [N/A] means that paper does not include experiments requiring code.
- Please see the NeurIPS code and data submission guidelines (<https://neurips.cc/public/guides/CodeSubmissionPolicy>) for more details.
- While we encourage the release of code and data, we understand that this might not be possible, so [No] is an acceptable answer. Papers cannot be rejected simply for not including code, unless this is central to the contribution (e.g., for a new open-source benchmark).
- The instructions should contain the exact command and environment needed to run to reproduce the results. See the NeurIPS code and data submission guidelines (<https://neurips.cc/public/guides/CodeSubmissionPolicy>) for more details.
- The authors should provide instructions on data access and preparation, including how to access the raw data, preprocessed data, intermediate data, and generated data, etc.
- The authors should provide scripts to reproduce all experimental results for the new proposed method and baselines. If only a subset of experiments are reproducible, they should state which ones are omitted from the script and why.
- At submission time, to preserve anonymity, the authors should release anonymized versions (if applicable).
- Providing as much information as possible in supplemental material (appended to the paper) is recommended, but including URLs to data and code is permitted.

6. Experimental setting/details

Question: Does the paper specify all the training and test details (e.g., data splits, hyperparameters, how they were chosen, type of optimizer) necessary to understand the results?

Answer: [Yes]

Justification: The Experimental Design section specifies the adversity axis and its sweep values, the ten arms, the episode protocol (20 profiles $\times$ 10 seeds $\times$ 10 arms $\times$ 6 points), and the profile-clustered statistical procedure. Appendix B lists every agent hyperparameter. The same section documents the quality gate applied at each sweep point and states that results are reported only over the passing range.

1060 Guidelines:

1061 • The answer [N/A] means that the paper does not include experiments.

1062 • The experimental setting should be presented in the core of the paper to a level of detail

1063 that is necessary to appreciate the results and make sense of them.

1064 • The full details can be provided either with the code, in appendix, or as supplemental

1065 material.

1066 **7. Experiment statistical significance**

1067 Question: Does the paper report error bars suitably and correctly defined or other appropriate

1068 information about the statistical significance of the experiments?

1069 Answer: [Yes]

1070 Justification: The design is within-subject (the same 100 profiles face every arm), so

1071 all inferential comparisons use paired t -tests on profile-level means ($n=100$), reporting

1072 the paired difference, its 95% confidence interval, and the paired effect size d_z . Claims

1073 resting on the absence of an effect are additionally supported by TOST equivalence tests

1074 against a prespecified smallest effect size of interest ($\delta=0.05$), rather than inferred from

1075 non-significance. Confidence intervals are reported for every headline contrast, including

1076 the crossover. The clustering unit and test are stated in the Experimental Design section.

1077 Guidelines:

1078 • The answer [N/A] means that the paper does not include experiments.

1079 • The authors should answer [Yes] if the results are accompanied by error bars, confidence

1080 intervals, or statistical significance tests, at least for the experiments that support the

1081 main claims of the paper.

1082 • The factors of variability that the error bars are capturing should be clearly stated (for

1083 example, train/test split, initialization, random drawing of some parameter, or overall

1084 run with given experimental conditions).

1085 • The method for calculating the error bars should be explained (closed form formula,

1086 call to a library function, bootstrap, etc.)

1087 • The assumptions made should be given (e.g., Normally distributed errors).

1088 • It should be clear whether the error bar is the standard deviation or the standard error

1089 of the mean.

1090 • It is OK to report 1-sigma error bars, but one should state it. The authors should

1091 preferably report a 2-sigma error bar than state that they have a 96% CI, if the hypothesis

1092 of Normality of errors is not verified.

1093 • For asymmetric distributions, the authors should be careful not to show in tables or

1094 figures symmetric error bars that would yield results that are out of range (e.g., negative

1095 error rates).

1096 • If error bars are reported in tables or plots, the authors should explain in the text how

1097 they were calculated and reference the corresponding figures or tables in the text.

1098 **8. Experiments compute resources**

1099 Question: For each experiment, does the paper provide sufficient information on the com-

1100 puter resources (type of compute workers, memory, time of execution) needed to reproduce

1101 the experiments?

1102 Answer: [Yes]

1103 Justification: A dedicated Compute Resources section states the machine (Intel Core i7-

1104 1165G7, 4 cores, 32 GB RAM, no GPU), the wall-clock time for the main sweep (60,000

1105 episodes in 31.4 s single-threaded), the coefficient search (3 min), and the per-decision

1106 cost (14 μ s), together with the operator's $O(n^2 + A)$ per-session arithmetic and its lack of

1107 gradients, value functions, or replay. Exploratory and discarded runs during development

1108 were of the same negligible per-run cost, and the entire project consumed less compute than

1109 a single GPU-minute.

1110 Guidelines:

1111 • The answer [N/A] means that the paper does not include experiments.

- The paper should indicate the type of compute workers CPU or GPU, internal cluster, or cloud provider, including relevant memory and storage.
- The paper should provide the amount of compute required for each of the individual experimental runs as well as estimate the total compute.
- The paper should disclose whether the full research project required more compute than the experiments reported in the paper (e.g., preliminary or failed experiments that didn't make it into the paper).

9. Code of ethics

Question: Does the research conducted in the paper conform, in every respect, with the NeurIPS Code of Ethics <https://neurips.cc/public/EthicsGuidelines>?

Answer: [Yes]

Justification: The research uses only a synthetic simulator with no human subjects, no personal data, and no scraped assets. Anonymity is preserved throughout the submission. Potential dual-use concerns arising from modeling learner internal states are discussed in the Impact Statement.

Guidelines:

- The answer [N/A] means that the authors have not reviewed the NeurIPS Code of Ethics.
- If the authors answer [No], they should explain the special circumstances that require a deviation from the Code of Ethics.
- The authors should make sure to preserve anonymity (e.g., if there is a special consideration due to laws or regulations in their jurisdiction).

10. Broader impacts

Question: Does the paper discuss both potential positive societal impacts and negative societal impacts of the work performed?

Answer: [Yes]

Justification: The Impact Statement section discusses the positive case (tutoring systems that respect cognitive and emotional limits, supported directly by the paper's viability results) and the negative case (the same mechanism inverted into an engagement-maximizing system that learns when to intervene to keep a user hooked), and identifies transparency about whose homeostasis is regulated as the mitigation.

Guidelines:

- The answer [N/A] means that there is no societal impact of the work performed.
- If the authors answer [N/A] or [No], they should explain why their work has no societal impact or why the paper does not address societal impact.
- Examples of negative societal impacts include potential malicious or unintended uses (e.g., disinformation, generating fake profiles, surveillance), fairness considerations (e.g., deployment of technologies that could make decisions that unfairly impact specific groups), privacy considerations, and security considerations.
- The conference expects that many papers will be foundational research and not tied to particular applications, let alone deployments. However, if there is a direct path to any negative applications, the authors should point it out. For example, it is legitimate to point out that an improvement in the quality of generative models could be used to generate Deepfakes for disinformation. On the other hand, it is not needed to point out that a generic algorithm for optimizing neural networks could enable people to train models that generate Deepfakes faster.
- The authors should consider possible harms that could arise when the technology is being used as intended and functioning correctly, harms that could arise when the technology is being used as intended but gives incorrect results, and harms following from (intentional or unintentional) misuse of the technology.
- If there are negative societal impacts, the authors could also discuss possible mitigation strategies (e.g., gated release of models, providing defenses in addition to attacks, mechanisms for monitoring misuse, mechanisms to monitor how a system learns from feedback over time, improving the efficiency and accessibility of ML).

11. Safeguards

Question: Does the paper describe safeguards that have been put in place for responsible release of data or models that have a high risk for misuse (e.g., pre-trained language models, image generators, or scraped datasets)?

Answer: [N/A]

Justification: The released artifact is a small simulation notebook implementing a synthetic environment and a two-drive regulator. It contains no pretrained model, no generative capability, and no data, and therefore poses no misuse risk requiring safeguards.

Guidelines:

- The answer [N/A] means that the paper poses no such risks.
- Released models that have a high risk for misuse or dual-use should be released with necessary safeguards to allow for controlled use of the model, for example by requiring that users adhere to usage guidelines or restrictions to access the model or implementing safety filters.
- Datasets that have been scraped from the Internet could pose safety risks. The authors should describe how they avoided releasing unsafe images.
- We recognize that providing effective safeguards is challenging, and many papers do not require this, but we encourage authors to take this into account and make a best faith effort.

12. Licenses for existing assets

Question: Are the creators or original owners of assets (e.g., code, data, models), used in the paper, properly credited and are the license and terms of use explicitly mentioned and properly respected?

Answer: [N/A]

Justification: No existing datasets, code packages, or models are used beyond standard open-source scientific libraries (scipy, BSD-3-Clause; matplotlib, PSF-based license), used in their ordinary capacity. The conceptual sources underlying the environment, notably the Rasch model and the Zone of Proximal Development, are cited in the references.

Guidelines:

- The answer [N/A] means that the paper does not use existing assets.
- The authors should cite the original paper that produced the code package or dataset.
- The authors should state which version of the asset is used and, if possible, include a URL.
- The name of the license (e.g., CC-BY 4.0) should be included for each asset.
- For scraped data from a particular source (e.g., website), the copyright and terms of service of that source should be provided.
- If assets are released, the license, copyright information, and terms of use in the package should be provided. For popular datasets, paperswithcode.com/datasets has curated licenses for some datasets. Their licensing guide can help determine the license of a dataset.
- For existing datasets that are re-packaged, both the original license and the license of the derived asset (if it has changed) should be provided.
- If this information is not available online, the authors are encouraged to reach out to the asset's creators.

13. New assets

Question: Are new assets introduced in the paper well documented and is the documentation provided alongside the assets?

Answer: [Yes]

Justification: The new asset is the simulation notebook, provided anonymized in the supplementary material. It is documented inline with markdown cells covering the research question, the preregistered predictions, the rationale for each design change, the quality gate criteria, and the prediction verdicts, and it emits a structured JSON of all results.

1218	Guidelines:
1219	• The answer [N/A] means that the paper does not release new assets.
1220	• Researchers should communicate the details of the dataset/code/model as part of their submissions via structured templates. This includes details about training, license, limitations, etc.
1221	
1222	
1223	• The paper should discuss whether and how consent was obtained from people whose asset is used.
1224	
1225	• At submission time, remember to anonymize your assets (if applicable). You can either create an anonymized URL or include an anonymized zip file.
1226	
1227	14. Crowdsourcing and research with human subjects
1228	Question: For crowdsourcing experiments and research with human subjects, does the paper include the full text of instructions given to participants and screenshots, if applicable, as well as details about compensation (if any)?
1229	
1230	
1231	Answer: [N/A]
1232	Justification: The paper involves no crowdsourcing and no human subjects. The “student” is a simulated agent in a synthetic environment; no data from real learners is collected or used.
1233	
1234	Guidelines:
1235	• The answer [N/A] means that the paper does not involve crowdsourcing nor research with human subjects.
1236	
1237	• Including this information in the supplemental material is fine, but if the main contribution of the paper involves human subjects, then as much detail as possible should be included in the main paper.
1238	
1239	
1240	• According to the NeurIPS Code of Ethics, workers involved in data collection, curation, or other labor should be paid at least the minimum wage in the country of the data collector.
1241	
1242	
1243	15. Institutional review board (IRB) approvals or equivalent for research with human subjects
1244	
1245	Question: Does the paper describe potential risks incurred by study participants, whether such risks were disclosed to the subjects, and whether Institutional Review Board (IRB) approvals (or an equivalent approval/review based on the requirements of your country or institution) were obtained?
1246	
1247	
1248	
1249	Answer: [N/A]
1250	Justification: No human subjects research was conducted, so no IRB approval or equivalent review was required.
1251	
1252	Guidelines:
1253	• The answer [N/A] means that the paper does not involve crowdsourcing nor research with human subjects.
1254	
1255	• Depending on the country in which research is conducted, IRB approval (or equivalent) may be required for any human subjects research. If you obtained IRB approval, you should clearly state this in the paper.
1256	
1257	
1258	• We recognize that the procedures for this may vary significantly between institutions and locations, and we expect authors to adhere to the NeurIPS Code of Ethics and the guidelines for their institution.
1259	
1260	
1261	16. Declaration of LLM usage
1262	Question: Does the paper describe the usage of LLMs if it is an important, original, or non-standard component of the core methods in this research? Note that if the LLM is used only for writing, editing, or formatting purposes and does <i>not</i> impact the core methodology, scientific rigor, or originality of the research, declaration is not required.
1263	
1264	
1265	
1266	Answer: [Yes]
1267	Justification: We disclose LLM usage for transparency although it does not constitute a core-method component. No language model participates in the agent, the environment, the baselines, or the analysis: all are explicit numerical simulations. LLM assistance was
1268	
1269	

1270 confined to (i) code assistance, using Devin as an IDE with Kimi 3 and DeepSeek v4 Pro,
1271 and Claude Opus 5 and Claude Fable 5, and (ii) writing and editing assistance, using Claude
1272 Opus 5 and Claude Fable 5. All experimental design decisions, preregistered predictions,
1273 statistical procedures, and interpretations are the authors' own, and every reported number
1274 was produced by executing the released notebook.

1275 Guidelines:

- 1276 • The answer [N/A] means that the core method development in this research does not
1277 involve LLMs as any important, original, or non-standard components.
- 1278 • Please refer to our LLM policy in the NeurIPS handbook for what should or should not
1279 be described.